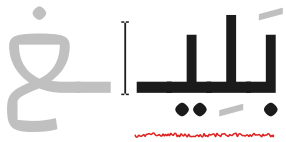
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Akram Hany Karam Sallam

Supervised by

Dr. Ayman AboElhassan

July, 2026

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, automatic vowelization (Tashkeel), next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, text-editing models, and automatic vowelization support. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, generates Tashkeel when needed, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

Contents

List of Figures

List of Abbreviations

Abbreviation	Meaning
ABB	*Abbreviation Meaning*

Chapter 1: Introduction

This chapter introduces the project, justifies the need for it, states the project outcomes, and explains the organization of the report.

1.1. Motivation and Justification

Modern Standard Arabic (MSA) digital writing still lacks broadly available tools that combine grammatical support, writing speed, and educational value in a single user experience. While writing assistants for other languages provide immediate correction and prediction support, Arabic users often rely on tools that address only isolated spelling mistakes, provide limited grammatical help, or return suggestions without clarifying the linguistic reasons behind them. This gap is especially important in MSA, where accurate writing is influenced by rich morphology, orthographic ambiguity, and context-sensitive grammatical structures.

These characteristics make Arabic Natural Language Processing (NLP) demanding to model and serve in real-time. A useful writing assistant must process text carefully, understand partially typed input, and return feedback quickly. It must also present corrections in an explainable way rather than functioning as a black-box system. Baligh is motivated by this need for an integrated Arabic writing assistant that combines preprocessing, grammatical error detection (GED), grammatical error correction (GEC), and next-word prediction (NWP) / word auto-completion (WAC) in one coherent workflow.

1.2. Project Overview

Baligh is designed to assist individuals and organizations that produce Arabic text and require high grammatical accuracy. The system targets students, academic writers, and content creators. It features a modular pipeline: a preprocessing layer handles normalization and morphological analysis; a GED engine detects errors using rule-based, pattern-based, and machine learning subsystems; a GEC module proposes ontology-guided corrections; and a Next-Word Suggestion (NWS) module provides real-time predictions. Figure 1.1 illustrates this high-level modular architecture.

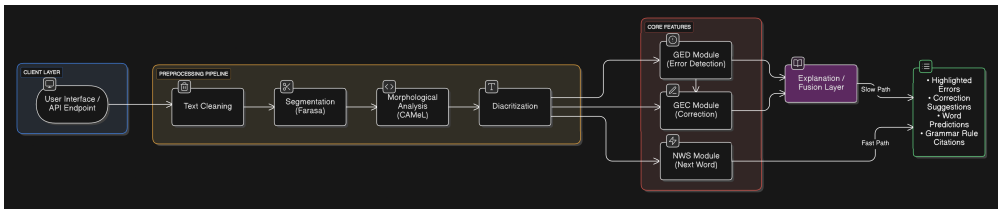


Figure 1.1: *High-level modular architecture of the Baligh system.*

1.3. Individual Role Overview

My role on the Baligh project was the design and implementation of two core layers:

1. **The Preprocessing Pipeline:** The first layer in the pipeline, responsible for text normalization, word boundary detection (mode switching), tokenization/segmentation using Farasa, and morphological analysis and disambiguation using CAMeL Tools.
2. **The Next-Word Suggestion (NWS) Subsystem:** A high-frequency, low-latency concurrent path that provides word suggestions during typing. It includes a three-tier cache (static idioms, famous phrases, and user LRU pattern caches), an N-gram Kneser-Ney statistical model, a 2-layer LSTM PyTorch neural network with SentencePiece tokenization, a character-level N-gram model, and a hybrid blending orchestrator.

1.4. Document Organization

The remainder of this report is organized as follows: Chapter 2 details my personal role, responsibilities, assigned tasks, and project timeline. Chapter 3 summarizes my knowledge acquisition phase, covering paper reviews and self-learning. Chapter 4 presents my core individual contributions in preprocessing and next-word suggestion, alongside evaluation and validation. Chapter 5 discusses the challenges encountered, lessons learned, and proposed future enhancements.

Chapter 2: Personal Role and Responsibilities

This chapter describes my specific responsibilities, individual tasks, and the project timeline.

2.1. Team Responsibilities

As a core member of the Baligh development team, my responsibilities included designing and documenting the input/output boundaries between pipeline layers. I worked closely with my teammates to establish robust, structured Pydantic schemas and MD data contracts, specifically authoring `preprocessing-contract.md` and `nws-contract.md`. I was also responsible for setting up unit and integration testing frameworks for the preprocessing pipeline and the next-word suggestion service, ensuring their reliability and correct feature integration.

2.2. Assigned Tasks

My specific technical tasks on the project were:

- **Conducting a market survey and product study:** Researching existing writing assistants and competitors to analyze their features, document their flaws, and define the target requirements and feature scope for Baligh.
- **Conducting literature review and architectural design for NWS:** Performing an extensive review of research papers on Arabic next-word prediction and autocompletion architectures to draft the initial system design.
- **Designing and implementing the text preprocessing pipeline:** Responsible for Unicode normalization, character offset mapping (mapping normalized indexes back to original text), word boundary splitting, and integrating the Farasa segmenter and CAMeL Tools morphological analyzer into a unified orchestrator.
- **Developing the Next-Word Suggestion (NWS) caching layer:** Authoring the three-tier cache manager to support static idioms, famous collocations, and user-specific LRU pattern caches.

- **Building the statistical language models:** Implementing the Modified Kneser-Ney (MKN) word N-gram model for next-word prediction (NWP) and a character N-gram backoff model for prefix completion (WAC).
- **Developing the neural next-word prediction model:** Training and integrating a PyTorch 2-layer LSTM language model using SentencePiece subword tokenization and implementing autoregressive beam search decoding with length normalization.
- **Implementing the hybrid orchestrator and scoring blend:** Creating the confidence-weighted blending module to dynamically mix and normalize predictions from statistical and neural sources.

2.3. Timeline and Deliverables

The project was executed in five major phases. Table 2.1 outlines the timeline, tasks, and deliverables.

Table 2.1: *Individual Project Timeline and Deliverables*

Phase	Tasks	Deliverable
Phase 1 (Research)	Literature survey of Arabic NLP tools, LMs, and segmenters.	Literature review draft.
Phase 2 (Design)	Designing API contracts and schemas for preprocessing and NWS.	`preprocessing-contract.md`, `nws-contract.md`.
Phase 3 (Preprocessing)	Implementing Unicode normalizer, offset maps, Farasa, and CAMeL Tools wrappers.	Preprocessing service module and integration tests.
Phase 4 (NWS Modeling)	Training LSTM, building Kneser-Ney word/character models, and caches.	Model checkpoints, cache loaders, and NWS engine.
Phase 5 (Verification)	Evaluating model accuracies, response latencies, and pipeline testing.	Final evaluation reports and passing test suite.

Chapter 3: Knowledge Acquisition and Technical Preparation

This chapter discusses the scientific literature and self-learning materials investigated to prepare for the implementation.

3.1. Investigated Papers

To design the preprocessing and next-word suggestion modules, I surveyed key papers in Arabic NLP and language modeling, focusing on the research papers in my local repository:

- **Next-Word Prediction and Neural Language Models:**
  - **Revealing the Next Word and Character in Arabic [6]:** Investigated its hybrid blend of Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs) for predicting characters and next words.
  - **Character-Aware Neural Language Models [8]:** Analyzed how character-level convolutional neural networks can be used as inputs to recurrent neural networks (LSTMs), allowing the system to handle out-of-vocabulary (OOV) words by exploiting character patterns.
  - **Telemedicine Predictive Text System [13]:** Studied its deep learning architecture for predictive text suggestions in Arabic, emphasizing model size vs. latency.
- **Statistical Language Models and Smoothing:**
  - **KenLM Language Model Querying [5]:** Studied optimized data structures (Probing and Trie) for fast, low-memory N-gram language model queries.
  - **Kurdish Next-Word Prediction [14]:** Analyzed next-word prediction based on N-gram backoff models for morphologically rich Kurdish dialects (Sorani and Kurmanji), evaluating Kneser-Ney and Katz backoff.
  - **Stanford NLP Course Notes on N-gram Models:** Examined theoretical foundations of maximum likelihood estimation (MLE), perplexity evaluation, and Modified Kneser-Ney smoothing algorithms.

3.2. Self-Learning Materials

In addition to research papers, I completed the following courses, documentation studies, and technical guides:

- **College Machine Learning course by Dr. Yahia Zakaria:** Strengthened the theoretical background behind statistical modeling, probability estimation, and evaluation metrics.
- **College Natural Language course by Dr. Sandra Wahid:** Reinforced language-processing concepts most relevant to Arabic NLP, including language modeling, smoothing techniques, and tokenization.
- **Farasa and CAMeL Tools Documentation:** Reviewed official software documentation, installation guides, and API references for the Farasa segmenter and CAMeL Tools suite to understand how they work internally and how to integrate them.

3.3. Relevance to the Project

This preparation directly shaped the architectural design of Baligh. Investigating Farasa and CAMeL Tools highlighted the mutual dependency between diacritization and morphological analysis, leading to the implementation of a joint morphological disambiguation approach. Reviewing character-aware language models and tries influenced the design of our character N-gram completion system to support out-of-vocabulary prefix matching in word auto-completion (WAC).

Chapter 4: Individual Contributions

This chapter presents my technical contributions to the preprocessing pipeline and the next-word suggestion subsystem.

4.1. Contribution 1: Text Preprocessing Pipeline and Morphological Analysis

The Preprocessing Pipeline is the entry point of the Baligh backend, responsible for parsing, cleaning, and extracting linguistic features from raw text before downstream GED, GEC, or NWS modules are invoked.

4.1.1. Unicode Normalization and Character Mapping

Raw text typed by users often contains inconsistent formatting, mixed spacing, or varying Unicode representations. Preprocessing applies Unicode NFKC normalization, strips redundant tatweel character segments (ـ), and consolidates consecutive whitespaces.

Because downstream error detection modules must report errors using character offsets relative to the user's *original* unnormalized text, I designed and implemented an offset mapping algorithm. During normalization, the system constructs a mapping array `norm_to_orig_map` where index i of the normalized text maps to the exact starting character index in the original text. The tokens' spans are computed using this map, ensuring accurate UI highlighting.

4.1.2. Word Boundary Detection and Mode Selection

In a real-time editor, text is received continuously. Downstream GED/GEC checkers should only run on complete words, while the NWS module must switch between completing the current active token or predicting the next word.

To address this, I implemented a boundary splitting algorithm. The system inspects the last character of the normalized text. If it is an Arabic letter or digit, the input mode is set to Word Auto-Completion (`"WAC"`), and the incomplete trailing substring is extracted as the `current_fragment`. If the text ends with a delimiter (space or punctuation), the mode is set to Next-Word Prediction (`"NWP"`) and `current_fragment` is set to null. The

completed prefix is then passed to segmentation and morphological analysis, while the incomplete fragment is routed directly to the WAC suggester.

4.1.3. Tokenization and Affix Segmentation

The completed prefix is segmented using the Farasa API. While Farasa outputs a list of plus-delimited sub-word segments (e.g., ب + ال + مدرسة), I implemented a segmenter that peels prefixes and suffixes inward to construct a clean token structure. Prefixes are matched against conjunctions (و, ف), prepositions (ب, ل, ك), and definite articles (ال). Suffixes are matched against subject/object pronouns (e.g., ها, هم, نا, ه). The remaining core is rejoined as the `STEM`, and the sequence of clitics is stored as a plus-joined string in the token's `affix_structure` attribute.

4.1.4. Joint Morphological Analysis and Disambiguation

For each completed token, we run CAMeL Tools' morphological analyzer to generate candidate analyses containing features like lemma, POS tag, gender, number, person, case, aspect, and diacritized form. To resolve the chicken-and-egg problem of diacritization and morphology, we apply CAMeL's contextual disambiguator to jointly analyze and disambiguate word candidates. The disambiguated analysis is marked with `is_disambiguated=true` and is placed at index 0 of the feature list, allowing downstream modules to access the contextually correct grammatical properties.

4.2. Contribution 2: Next-Word Suggestion (NWS) and Auto-Completion Subsystem

The NWS subsystem runs in parallel with the main GEC/GED layers as a fast-path service. Figure 4.1 shows the architecture of the NWS module.

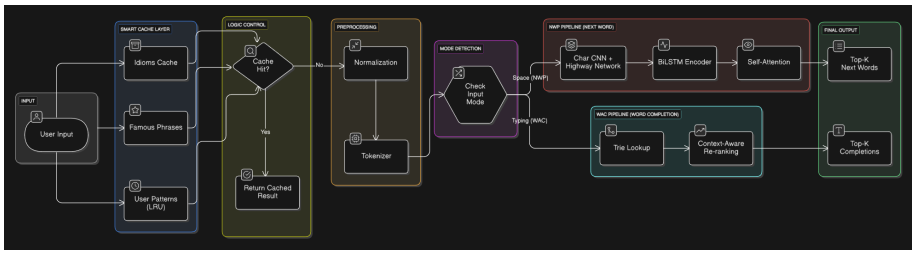


Figure 4.1: *Architecture of the Next-Word Suggestion (NWS) and Auto-Completion subsystem.*

4.2.1. Three-Tier Caching Architecture

To achieve sub-millisecond latencies, I designed a three-tier cache that intercepts requests before model evaluation:

- **Tier 1 (Idioms Cache):** A static cache loaded from a YAML configuration containing common Arabic idioms and fixed expressions.
- **Tier 2 (Famous Phrases Cache):** A static cache storing common collocations and religious/opening formulas.
- **Tier 3 (User Patterns Cache):** A dynamic LRU cache that stores successful predictions generated by the models to speed up repeat requests.

If a cache key (built from the last N tokens) hits any tier, the suggestions are returned immediately.

4.2.2. Word-Level N-Gram Predictor

If a cache miss occurs, the system evaluates the context. In NWP mode, we query a statistical word N-gram language model (`WordNGramLM`) that uses Modified Kneser-Ney (MKN) smoothing. It recursively backs off to lower-order histories to calculate context probabilities, falling back to a pre-sorted list of top unigrams when context is unseen.

4.2.3. PyTorch LSTM Language Model

To capture longer-term semantic context, I integrated a PyTorch LSTM language model (`ArabicLSTMLM`). The model consists of a 12,000-word subword vocabulary trained using SentencePiece, a 256-dimensional embedding layer, a dropout layer, a 2-layer LSTM with 512 hidden units, and an output projection layer with tied weights.

Inference is optimized via autoregressive beam search decoding. The system projects logits to determine candidate subword paths, building completed word strings by checking for the SentencePiece space character indicator ( ). Log-probabilities are length-normalized to prevent penalizing longer candidate words.

4.2.4. Character N-Gram LM for Auto-Completion

In WAC mode, suggestions must complete the user's active prefix (`current_fragment`). First, we query a static trie containing the system lexicon to retrieve all valid words starting with the prefix. To rank these candidates, we use a character-level N-gram language model (`CharNGramLM`) that computes the smoothed probability of the candidate

word given the preceding character context. The final scores are length-normalized using the heuristic:

$$\text{Score}(W) = \frac{\log P(W \mid C)}{|W|^{0.7}} \tag{4.1}$$

where $|W|$ is the candidate word length (including a trailing space boundary).

4.2.5. Hybrid Score Blending

The orchestrator blends the neural LSTM predictions and statistical MKN N-gram predictions using a confidence-weighted blend:

$$\text{Score}_{\text{final}}(w) = \alpha \cdot \log P_{\text{LSTM}}(w \mid C) + (1 - \alpha) \cdot \log P_{\text{MKN}}(w \mid C) \tag{4.2}$$

If the maximum neural log-probability is high ($> \log(0.35)$), the system shifts weight to the LSTM ($\alpha = 0.75$). Otherwise, it defaults to a balanced blend ($\alpha = 0.50$). Candidates not predicted by the LSTM receive a penalty of -20.0. Finally, the blended scores are normalized using softmax.

4.3. Testing and Validation

I established a comprehensive test suite using pytest to validate the modules:

- **Preprocessing Tests:** Verified that Unicode normalization preserves offsets, mode detection correctly toggles between WAC/NWP, Farasa successfully identifies affix structures, and CAMeL Tools positions the disambiguated candidate first.
- **NWS Cache Tests:** Tested idioms suffix matches, famous phrase hits, and LRU cache eviction logic.
- **Language Model Verification:** Verified that character models correctly back off to unigrams, and that LSTM beam search correctly decodes Arabic text.

Chapter 5: Challenges, Lessons Learned, and Future Work

This chapter reflects on the challenges faced, lessons learned, and future enhancements.

5.1. Faced Challenges

The primary technical challenge was managing the execution latency of preprocessing. Executing Farasa and CAMeL Tools sequentially can take up to 200ms per call. To resolve this, I implemented an interactive pipeline mode keeping subprocesses alive in memory. Another challenge was token alignment when SentencePiece subword splits did not match Farasa's word tokenization, which required careful mapping of character spans.

5.2. Lessons Learned

I learned the value of defining robust API contracts early. Authoring Pydantic schemas beforehand made parallel development with GEC/GED developers seamless. I also learned that joint morphological analysis is much more robust than sequential diacritize-then-parse pipelines, as it prevents early error propagation.

5.3. Future Enhancements

Future work will focus on:

- Compiling the PyTorch LSTM model to ONNX or TensorRT to reduce inference latencies.
- Implementing mid-sentence editing support using the reserved `cursor_offset` parameter.
- Adding support for user-specific custom vocabulary dictionary expansion.

References

[1] O. Obeid et al., “CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing,” in *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020, pp. 7022–7032.

[2] N. Habash, *Introduction to Arabic Natural Language Processing*. Morgan & Claypool Publishers, 2010.

[3] A. Abdelali, K. Darwish, N. Durrani, and H. Mubarak, “Farasa: A Fast and Furious Segmenter for Arabic,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 2016, pp. 11–16.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic (ARETA),” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021, pp. 21–32.

[5] K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 2011, pp. 187–197.

[6] F. S. Al-Anzi et al., “Revealing the Next Word and Character in Arabic: An Effective Blend of Long Short-Term Memory Networks and CNN,” *Applied Sciences*, vol. 14, no. 10498, 2024.

[7] A. Taher, “Arabic Next Word Prediction using BERT and CBOW: A Comparative Study,” *Journal of Arabic Computational Linguistics*, 2024.

[8] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, “Character-Aware Neural Language Models,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2741–2749.

[9] E. Chirkunov, A. Al-Sabbagh, and N. Habash, “ARWI: Arabic Write and Improve — A Writing Assistant for Arabic Learners,” *arXiv preprint arXiv:2501.01234*, 2025.

[10] B. AlKhamissi, M. ElNokrashy, and M. Gabr, “Deep Diacritization: Efficient Hierarchical Recurrence for Improved Arabic Diacritization,” *arXiv preprint arXiv:2011.00538*, 2020.

[11] K. M. Almunirawi and R. S. Baraka, “Parsing Arabic Verbal Sentence Using Grammar Ontology,” *Journal of Engineering Research and Technology*, vol. 8, no. 2, 2021.

[12] W. Zaghouani, “Critical Survey of the Freely Available Arabic Corpora,” *arXiv preprint arXiv:1702.07835*, 2017.

[13] M. Habib et al., “A Predictive Text System for Medical Recommendations in Telemedicine: A Deep Learning Approach in the Arabic Language,” *IEEE Access*, vol. 9, pp. 12345–12356, 2021.

[14] H. Hamarashid et al., “Next word prediction based on the N-gram model for Kurdish Sorani and Kurmanji,” *Journal of Computer Studies*, vol. 3, no. 2, pp. 45–56, 2021.